Algorithms for Approximate Inversion of Weave Structures

Peter E. Francis

May 4, 2026

Abstract

This paper studies the inverse problem for shaft-loom weaving: given a desired binary drawdown, find a threading, tie-up, and treadling that reproduce it exactly when possible, and approximate it well when exact reproduction is impossible under loom constraints.

Contents

1 Introduction **2**

2 Exact Solvability **5**

3 Alpha Algorithms (Hamming Error) **8**

3.1 Alpha 1 8

3.2 Alpha 2 9

3.3 Alpha 3 11

3.4 Benchmarks 12

1 Introduction

A single-layer woven structure may be idealized as a binary matrix

$$\widehat{D} \in \{0,1\}^{p\times w},$$

where p is the number of weft picks and w is the number of warp ends. Rows index picks and columns index warp ends. We use the convention

$$\widehat{D}_{ij} = \begin{cases} 1, & \text{warp end } j \text{ passes over weft pick } i, \\ 0, & \text{weft pick } i \text{ passes over warp end } j. \end{cases}$$

In this language, a Jacquard loom is essentially unconstrained: each warp end may be controlled independently on each pick, so every finite binary matrix is attainable, ignoring physical considerations such as yarn behavior, abrasion, float length, and cloth stability. Matrix and algebraic models of loom constraints go back at least to Lourie's work on loom-constrained designs, which represents a weave as a binary matrix and studies how to recover loom data from that matrix [1].

The situation is very different for a shaft loom. A warp end is threaded through one shaft, and all warp ends on the same shaft move together. Thus a shaft loom cannot directly control individual warp ends; it controls equivalence classes of warp ends. Let t be the number of treadles, s the number of shafts, and r the maximum number of treadles allowed to be pressed on a single pick. We encode a draft by three binary matrices:

$$A \in \{0,1\}^{p\times t}, \qquad B \in \{0,1\}^{s\times t}, \qquad C \in \{0,1\}^{s\times w}.$$

Here A is the treadling matrix, with $A_{i\ell} = 1$ if treadle ℓ is pressed on pick i; B is the tie-up matrix, with $B_{h\ell} = 1$ if treadle ℓ lifts shaft h; and C is the threading matrix, with $C_{hj} = 1$ if warp end j is threaded on shaft h. Each column of C contains exactly one nonzero entry, and each row of A contains at most r nonzero entries.

The natural product in this notation is

$$D = AB^TC.$$

The matrix D is not merely binary. It records the redundancy with which each cell is activated.

Theorem 1 (Redundancy matrix). *Let $A \in \{0,1\}^{p\times t}$ be a treadling matrix, let $B \in \{0,1\}^{s\times t}$ be a tie-up matrix, and let $C \in \{0,1\}^{s\times w}$ be a threading matrix with exactly one nonzero entry in each column. Define*

$$D = AB^TC.$$

Then $D \in \mathbb{Z}_{\geq 0}^{p\times w}$, and D_{ij} is the number of pressed treadles on pick i that lift the shaft containing warp end j. In particular, if

$$\widehat{D}_{ij} = \mathbf{1}\{D_{ij} > 0\},$$

then $\widehat{D}_{ij} = 1$ exactly when warp end j is lifted over pick i. Moreover, if at most r treadles are pressed on each pick, then

$$0 \leq D_{ij} \leq r \qquad \text{for all } i,j.$$

Proof. By the definition of matrix multiplication,

$$D_{ij} = (AB^TC)_{ij} = \sum_{\ell=1}^{t}\sum_{h=1}^{s} A_{i\ell}B_{h\ell}C_{hj}.$$

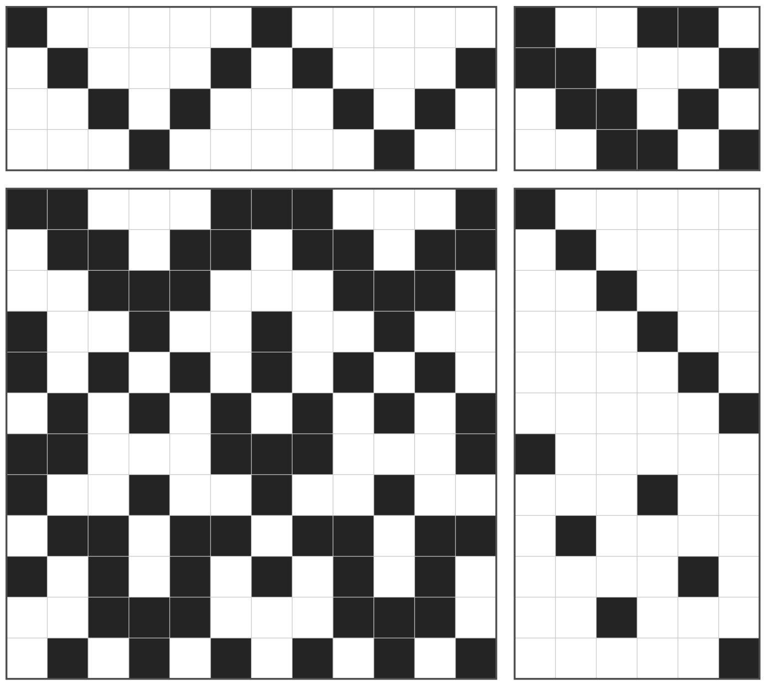

Figure 1: A draft written as matrices. Dark cells are entries equal to 1. This example uses $w = p = 12$, $s = 4$, and $t = 6$: the threading C, tie-up B, and treadling A determine the drawdown by thresholding the redundancy matrix $D = AB^T C$.

Since warp end j is threaded on exactly one shaft, there is a unique shaft $h(j)$ such that $C_{h(j),j} = 1$, and $C_{hj} = 0$ for $h \neq h(j)$. Therefore the double sum collapses to

$$D_{ij} = \sum_{\ell=1}^{t} A_{i\ell} B_{h(j),\ell}.$$

For a fixed pick i, the factor $A_{i\ell}$ is 1 exactly when treadle ℓ is pressed. The factor $B_{h(j),\ell}$ is 1 exactly when treadle ℓ lifts the shaft $h(j)$ carrying warp end j. Thus each summand is 1 exactly for a pressed treadle that lifts the shaft containing warp end j, and is 0 otherwise. Hence D_{ij} counts those treadles.

It follows that $D_{ij} > 0$ exactly when at least one pressed treadle lifts the shaft containing warp end j. This is precisely the event that warp end j is lifted over pick i, so the binary drawdown is $\widehat{D}_{ij} = \mathbf{1}\{D_{ij} > 0\}$. Finally, if at most r treadles are pressed on pick i, then at most r terms in $\sum_{\ell=1}^{t} A_{i\ell} B_{h(j),\ell}$ can be nonzero, so $D_{ij} \leq r$. □

This is not ordinary unconstrained matrix factorization; it is a discrete factorization through treadle, shaft, and warp-thread incidence matrices. Given a target drawdown $D^* \in \{0,1\}^{p \times w}$, the exact inverse problem is to decide whether there exist admissible matrices A, B, C such that

$$D^*_{ij} = \mathbf{1}\{(AB^T C)_{ij} > 0\}.$$

When no such draft exists under the bounds s, t, r, the approximate inverse problem is to choose admissible draft matrices that minimize an objective

$$\Phi(\widehat{D}, D^*; A, B, C), \qquad \widehat{D}_{ij} = \mathbf{1}\{(AB^TC)_{ij} > 0\}.$$

The functional Φ describes the criterion by which an approximation is judged.

A natural first objective, and the one used by the Alpha algorithms below, is Hamming error $\Phi = \mathrm{err}$,

$$\mathrm{err}(\widehat{D}, D^*) = \#\{(i,j) : \widehat{D}_{ij} \neq D^*_{ij}\},$$

but the search process is still sensitive to where errors occur. Even when two candidate drafts have the same Hamming error, a wrong cell that changes a rare row behavior, a rare column behavior, or a local boundary may affect the subsequent co-clustering differently from a wrong cell in a large homogeneous region. Alpha 2 and Alpha 3 below treat such quantities as heuristics for guiding the optimization, not as changes to the objective Φ. Separate objectives could also penalize long warp or weft floats, disconnected prefabric behavior, or other non-clothlike structures, but those are not part of the Alpha objective here.

The purpose of this paper is to develop and compare algorithms for approximate inversion. The central questions are:

(i) how to find exact factorizations when they exist;

(ii) how to search efficiently when the target exceeds the available loom constraints;

(iii) how cell-impact and redundancy heuristics can guide the search while retaining Hamming error as the primary objective;

(iv) how to incorporate clothlike constraints such as float tolerance; and

(v) how to test an inversion engine using targets known to be generated by random drafts as well as targets generated by random binary drawdowns.

History and context

There are two closely related mathematical traditions behind this note. The first is the algebraic and algorithmic analysis of weave drafts. Lourie's "loom-constrained designs" paper formulates the problem of determining a loom mechanism, threading-like initial conditions, and dynamic control information from a binary representation of a weave [1]. Hoskins and Hoskins studied matrix equations and factoring algorithms arising from structural analysis of woven fabrics [2, 3], while later work on binary interlacement arrays and bounded-float twills develops related finite combinatorial models [4, 5].

The second tradition is the geometry and symmetry theory of fabrics and prefabrics. Grünbaum and Shephard introduced the modern mathematical treatment of satins, twills, prefabrics, and isonemal fabrics [6, 7, 8, 9]. Roth classified the symmetry groups of periodic isonemal fabrics [10], and Thomas refined Roth's taxonomy for prefabrics with parallel axes, perpendicular axes, and no axes of symmetry [11, 12, 13]. Questions about whether a prefabric "hangs together" are separate from symmetry classification and lead to their own graph-like connectivity criteria [14, 15, 16].

The present paper uses the same binary-matrix language, but focuses on a slightly different computational question: when an exact shaft-loom factorization is unavailable or undesirable, how should one approximate the target drawdown? In particular, we keep track of the integer redundancy matrix $D = AB^TC$, allow multiple treadles to be pressed on a pick, and compare search heuristics and secondary diagnostics that may value visual locality, motif continuity, or clothlike float behavior differently from raw Hamming error.

2 Exact Solvability

Proposition 1 (Finite shaft-loom attainability with multiple treadles)**.** *Let $E \in \{0,1\}^{p\times w}$ be a finite binary drawdown. Delete duplicate columns of E, and call the resulting matrix E_{col}. Let s_0 be the number of columns of E_{col}, and let $\mathcal{R} \subseteq \{0,1\}^{s_0}$ be the set of distinct rows of E_{col}.*

For a set $\mathcal{B} \subseteq \{0,1\}^{s_0}$, define its r-fold union closure by

$$\mathcal{U}_r(\mathcal{B}) = \{b_1 \vee b_2 \vee \cdots \vee b_k : 0 \le k \le r,\ b_1,\ldots,b_k \in \mathcal{B}\},$$

where $\vee$ denotes coordinatewise Boolean OR and the empty union is the zero vector. Then E is attainable on a shaft loom with at most s shafts, t treadles, and at most r treadles pressed on any pick if and only if

$$s_0 \le s \qquad \text{and} \qquad \mathcal{R} \subseteq \mathcal{U}_r(\mathcal{B})$$

for some set $\mathcal{B} \subseteq \{0,1\}^{s_0}$ with $|\mathcal{B}| \le t$.

When $r = 1$ and each pick is required to use a single treadle, this reduces to the simpler criterion that the reduced drawdown has at most s distinct column types and at most t distinct row types. With enough treadles or with a dobby-style liftplan, the minimum number of shafts is the number of distinct warp-column behaviors. For an infinite periodic prefabric, the same criterion applies to a fundamental period block.

Proof. First suppose that E is produced by a shaft loom with at most s shafts, t treadles, and at most r treadles pressed on a pick. If two warp ends are threaded on the same shaft, then they are lifted on exactly the same picks, so their columns in E are identical. Therefore the number of distinct column behaviors is at most the number of shafts, hence $s_0 \le s$.

After duplicate columns are removed, each treadle determines a binary vector in $\{0,1\}^{s_0}$: its entries record which of the s_0 column types are lifted by that treadle. Let $\mathcal{B}$ be the set of these treadle vectors. Then $|\mathcal{B}| \le t$. If pick i presses treadles $\ell_1,\ldots,\ell_k$, with $k \le r$, then its reduced row is the Boolean union of the corresponding treadle vectors,

$$b_{\ell_1} \vee \cdots \vee b_{\ell_k}.$$

Thus every distinct reduced row lies in $\mathcal{U}_r(\mathcal{B})$.

Conversely, suppose $s_0 \le s$ and $\mathcal{R} \subseteq \mathcal{U}_r(\mathcal{B})$ for some $\mathcal{B} = \{b_1,\ldots,b_m\} \subseteq \{0,1\}^{s_0}$ with $m \le t$. Assign each distinct column type to one shaft, threading every warp end with that column behavior through the corresponding shaft. Assign the vector b_ℓ to treadle ℓ, tying it to exactly those shafts whose column types have value 1 in b_ℓ. Since every reduced row of E is a Boolean union of at most r vectors from $\mathcal{B}$, choose one such representation for each pick and press the corresponding treadles. The resulting thresholded drawdown is exactly E.

For $r = 1$, each reduced row must itself be one of the treadle vectors, so the criterion reduces to counting distinct reduced row types. If pressing no treadle is disallowed, one simply omits the empty union from $\mathcal{U}_r(\mathcal{B})$.

For a periodic prefabric, all rows and columns are determined by a finite fundamental period block, so applying the same argument to that block gives the periodic statement. □

Thus, when $r > 1$, the minimum number of treadles is no longer simply the number of distinct reduced row types. It is the size of the smallest family of basic treadle lift patterns whose Boolean unions of size at most r cover all reduced row types.

Exact recovery algorithm

The proposition gives a direct complete algorithm for recovering a draft (A, B, C) from a target drawdown $\widehat{D}$, if such a draft exists under the size constraints. The algorithm starts from $\mathcal{R}$, because after duplicate warp-column behaviors are removed, $\mathcal{R}$ is equivalent to the target for the purpose of deciding which treadle lift patterns are needed. Repeated picks only repeat rows of A, and repeated warp-column behaviors only repeat columns of C.

Input. A target drawdown $\widehat{D} \in \{0,1\}^{p\times w}$, a shaft bound s, a treadle bound t, and a maximum number r of treadles pressed on a pick.

Step 1: recover the threading. Group the columns of $\widehat{D}$ by equality. Let

$$c(j) \in \{1, \dots, s_0\}$$

be the column class of warp end j, and let s_0 be the number of distinct column classes. If $s_0 > s$, stop: no draft with at most s shafts exists. Otherwise define $C \in \{0,1\}^{s\times w}$ by

$$C_{hj} = \begin{cases} 1, & h = c(j), \\ 0, & \text{otherwise.} \end{cases}$$

Unused shafts $s_0 + 1, \dots, s$ are left empty.

Step 2: reduce to row types. Delete duplicate column classes from $\widehat{D}$ to obtain

$$E_{\text{col}} \in \{0,1\}^{p\times s_0}.$$

Let $\mathcal{R} \subseteq \{0,1\}^{s_0}$ be the set of distinct rows of E_{col}. For each pick i, record its reduced row type

$$q(i) \in \mathcal{R}.$$

At this point the remaining problem is equivalent to finding treadle generators whose Boolean unions produce the rows in $\mathcal{R}$.

Step 3: solve the generator problem. Form the finite candidate set

$$\mathcal{C} = \{b \in \{0,1\}^{s_0} : b \le q \text{ coordinatewise for some } q \in \mathcal{R}\}.$$

Search over subsets $\mathcal{B} \subseteq \mathcal{C}$ with $|\mathcal{B}| \le t$. For each candidate $\mathcal{B}$, compute $\mathcal{U}_r(\mathcal{B})$. If

$$\mathcal{R} \subseteq \mathcal{U}_r(\mathcal{B}),$$

then choose, for every row type $q \in \mathcal{R}$, a representation

$$q = b_{\ell_1} \vee \cdots \vee b_{\ell_k}, \qquad k \le r, \qquad b_{\ell_1}, \dots, b_{\ell_k} \in \mathcal{B}.$$

If no such $\mathcal{B}$ exists, stop: no draft with the prescribed (s, t, r) constraints exists.

Step 4: recover the tie-up and treadling. Index the selected generators as

$$\mathcal{B} = \{b_1, \ldots, b_m\}, \qquad m \le t.$$

Define $B \in \{0,1\}^{s \times t}$ by

$$B_{h\ell} = \begin{cases} (b_\ell)_h, & 1 \le h \le s_0,\ 1 \le \ell \le m, \\ 0, & \text{otherwise.} \end{cases}$$

For each pick i, use the chosen representation of $q(i)$ and set

$$A_{i\ell} = \begin{cases} 1, & b_\ell \text{ is used in the chosen representation of } q(i), \\ 0, & \text{otherwise.} \end{cases}$$

Then each row of A has at most r nonzero entries.

Proposition 2 (Correctness of exact recovery)**.** *The algorithm above returns matrices A, B, C satisfying*

$$\widehat{D}_{ij} = \mathbf{1}\{(AB^T C)_{ij} > 0\}$$

whenever such matrices exist under the constraints s, t, r. If the algorithm reports failure, no such draft exists.

Proof. Step 1 is forced: warp ends with different columns in $\widehat{D}$ cannot be threaded on the same shaft, since shafts move their warp ends together. Thus any exact draft must use at least s_0 shafts, and grouping equal columns loses no information.

After this reduction, each treadle corresponds to a vector in $\{0,1\}^{s_0}$, recording which column classes it lifts. If a treadle vector appears in a union producing a row q, then it must be coordinatewise contained in q. Therefore every useful treadle vector lies in the candidate set $\mathcal{C}$; vectors outside $\mathcal{C}$ either cannot be used or would lift a shaft in a row where it should remain unlifted.

Thus an exact draft exists precisely when there is a set $\mathcal{B} \subseteq \mathcal{C}$ of at most t treadle vectors such that every reduced row type in $\mathcal{R}$ is a Boolean union of at most r vectors from $\mathcal{B}$. Step 3 searches this finite space exhaustively, so it finds such a family if one exists and fails only if none exists.

Given such a family, Step 4 makes B from the selected generators and makes A from the chosen union representation of each pick. By construction, the thresholded product $AB^T C$ reproduces every reduced row on the corresponding column classes, and C then repeats those column classes across the original warp ends. Therefore the thresholded drawdown is exactly $\widehat{D}$. $\square$

The periodic version of this statement uses the language of prefabrics and fabrics, which we recall here.

Definition 1 (Prefabric)**.** *A* prefabric *is a formal infinite over-under assignment for two transverse families of strands, one warp-like and one weft-like. A prefabric need not hang together as a physical cloth; it may fall apart into independent layers, strips, or components [6, 14, 16].*

Definition 2 (Fabric and isonemal fabric)**.** *A* fabric *is a prefabric whose strands interlace sufficiently to hang together. A fabric is* isonemal *if its symmetry group acts transitively on the set of strands; equivalently, for any two strands, there is a symmetry of the fabric that sends one strand to the other [6, 9].*

The symmetry theory of periodic fabrics, including isonemal fabrics, is largely orthogonal to shaft attainability. A design can be highly symmetric and still require many shaft-column behaviors. Conversely, a design can be attainable on few shafts while failing to be a good physical cloth because of floats, instability, or other textile constraints.

3 Alpha Algorithms (Hamming Error)

The Alpha algorithms are the approximate engines obtained by instantiating Φ with the Hamming weight of the binary discrepancy pattern $\Delta(\widehat{D}, D^*) = \widehat{D} \oplus D^*$. For Alpha 1, Alpha 2, and Alpha 3, the primary objective is the raw Hamming weight $\sum_{i,j} \Delta_{ij}$, so every cell disagreement has equal final cost. The difference between the methods is algorithmic rather than objective: Alpha 1 is the baseline co-clustering search, Alpha 2 adds cell-impact search pressure, and Alpha 3 adds redundancy search pressure. The later variants are not defined merely to be non-worsening wrappers around the earlier ones. They are allowed to change the search dynamics, and their justification is empirical: they should lower the average optimality gap under the same final Hamming objective.

3.1 Alpha 1

When the exact problem fails under the prescribed bounds s, t, r, the first approximation method, called *Alpha 1*, replaces the union-closure search by a finite binary co-clustering problem. Alpha 1 is the baseline member of the Alpha family: it uses ordinary Hamming disagreement throughout the co-clustering search and should be regarded as the reference approximation algorithm.

Let

$$\tau : \{1, \dots, p\} \to \{1, \dots, t'\}, \qquad \chi : \{1, \dots, w\} \to \{1, \dots, s'\}$$

be row and column assignments, with $t' \le t$ and $s' \le s$. The map τ assigns picks to provisional treadle states, while χ assigns warp ends to provisional shaft states. For a reduced tie-up

$$U \in \{0,1\}^{t' \times s'},$$

the corresponding approximate drawdown is

$$\widehat{D}_{ij} = U_{\tau(i),\chi(j)}.$$

Alpha 1 approximately minimizes the co-clustering loss

$$L_0(\tau, \chi, U) = \sum_{i=1}^{p} \sum_{j=1}^{w} \mathbf{1}\{U_{\tau(i),\chi(j)} \neq D^*_{ij}\}.$$

This objective is not the original shaft-loom problem, because it ignores the multi-treadle union representation while the clustering is being optimized. It is instead a tractable relaxation that is subsequently converted back into draft matrices A, B, C.

Proposition 3 (Majority tie-up update)**.** *Fix assignments τ and χ. A minimizer of $L_0(\tau, \chi, U)$ over $U \in \{0,1\}^{t' \times s'}$ is obtained entrywise by*

$$U_{ab} = \begin{cases} 1, & \displaystyle\sum_{i:\tau(i)=a} \sum_{j:\chi(j)=b} D^*_{ij} \ge \frac{1}{2} \#\{(i,j) : \tau(i) = a,\ \chi(j) = b\}, \\ 0, & \textit{otherwise.} \end{cases}$$

Proof. For fixed a and b, the entry U_{ab} affects only the block of cells (i, j) satisfying $\tau(i) = a$ and $\chi(j) = b$. Choosing $U_{ab} = 0$ incurs exactly one error for each 1 in that block, while choosing $U_{ab} = 1$ incurs exactly one error for each 0 in that block. Hence the optimal value is the majority value in the block. The blocks are disjoint, so the same argument applies independently to every entry of U. □

Procedure

Alpha 1 is a multi-start alternating minimization procedure. It uses a finite set $\mathcal{I}$ of initializations, such as farthest-first, density-ordered, and random assignments. The distance-based initializations use ordinary Hamming distances between rows and columns of D^*. For each choice of $s' \le s$, $t' \le t$, and initialization, Alpha 1 performs the following steps.

A1.1. Initialize

$$\tau : \{1, \dots, p\} \to \{1, \dots, t'\}, \qquad \chi : \{1, \dots, w\} \to \{1, \dots, s'\}.$$

A1.2. With τ and χ fixed, set U by the majority rule in the preceding proposition.

A1.3. With χ and U fixed, update each row assignment independently:

$$\tau(i) \in \operatorname*{argmin}_{1 \le a \le t'} \sum_{j=1}^{w} \mathbf{1}\{U_{a,\chi(j)} \neq D^*_{ij}\}.$$

A1.4. With τ and U fixed, update each column assignment independently:

$$\chi(j) \in \operatorname*{argmin}_{1 \le b \le s'} \sum_{i=1}^{p} \mathbf{1}\{U_{\tau(i),b} \neq D^*_{ij}\}.$$

A1.5. Recompute U. Stop if the triple (τ, χ, U) has stabilized, or if a prescribed iteration limit has been reached; otherwise return to Step A1.3.

For fixed s', t', and initialization, each update step weakly decreases L_0. Since the assignment space is finite, any run that forbids empty retained groups eventually reaches a local optimum with respect to these coordinate moves. The resulting reduced triple is converted into draft matrices by assigning column clusters to shafts and row clusters, or unions of row-cluster generators, to treadles. Candidate drafts are ranked first by Hamming error, then by draft size and secondary structural diagnostics such as line mismatch, blurred-field mismatch, density mismatch, and float behavior.

3.2 Alpha 2

Alpha 2 uses the same co-clustering framework and the same final objective $\Phi = \mathrm{err}$ as Alpha 1, but it is allowed to change the search dynamics. The motivating suggestion is that two single-cell changes with the same Hamming cost can have different algorithmic consequences: changing one cell may merge a rare row behavior into a common row behavior, alter a rare column behavior, or disturb a local boundary, while changing another cell may leave the row-column organization nearly unchanged. Alpha 2 uses this information as search pressure, not as a replacement for Φ.

Unlike a conservative ensemble version that keeps the full Alpha 1 candidate pool, the main Alpha 2 algorithm is not guaranteed to be no worse than Alpha 1 on every instance. Its purpose is to find better candidates on average by escaping different local optima. Final reporting and final candidate selection still use ordinary Hamming error first.

Cell-impact scores

For each row i, let

$$\rho_i \in \{0,1\}^w$$

be the i-th row pattern, and for each column j, let

$$\kappa_j \in \{0,1\}^p$$

be the j-th column pattern. Let $m_{\text{row}}(\rho)$ denote the number of times row pattern ρ occurs, and let $m_{\text{col}}(\kappa)$ denote the number of times column pattern κ occurs.

For a cell (i,j), let $\rho_i^{(j)}$ be the row pattern obtained by flipping coordinate j, and let $\kappa_j^{(i)}$ be the column pattern obtained by flipping coordinate i. Define

$$M_{ij} = \max\{m_{\text{row}}(\rho_i^{(j)}), m_{\text{col}}(\kappa_j^{(i)})\}.$$

The quantity M_{ij} measures whether flipping a cell would merge the affected row or column with an already existing behavior. Define also

$$\sigma_{ij} = \mathbf{1}\{m_{\text{row}}(\rho_i) = 1\} + \mathbf{1}\{m_{\text{col}}(\kappa_j) = 1\},$$

which records whether the cell lies in a row or column behavior that occurs only once, and let e_{ij} be the number of horizontal or vertical neighbors of (i,j) whose value differs from D^*_{ij}. Alpha 2 assigns the heuristic impact score

$$\eta_{ij} = \min\left\{6, 1 + \log_2(1 + M_{ij}) + 0.35\,\sigma_{ij} + 0.08\,e_{ij} + \frac{M_{ij}}{\max(p,w)}\right\}.$$

The score η_{ij} is not part of Φ. It is a heuristic estimate of how much row-column structure is touched by changing cell (i,j). The constants in this expression should be regarded as tunable, and the usefulness of the score is an empirical question rather than a theorem.

Impact-guided search pressure

Alpha 2 searches the same unweighted co-clustering relaxation

$$L_0(\tau,\chi,U) = \sum_{i=1}^{p}\sum_{j=1}^{w} \mathbf{1}\{U_{\tau(i),\chi(j)} \neq D^*_{ij}\}.$$

It also tracks the impact residual

$$G(\tau,\chi,U) = \sum_{i=1}^{p}\sum_{j=1}^{w} \eta_{ij}\,\mathbf{1}\{U_{\tau(i),\chi(j)} \neq D^*_{ij}\}.$$

During search, Alpha 2 may use either lexicographic ordering (L_0, G), or an annealed score

$$S_\lambda(\tau,\chi,U) = L_0(\tau,\chi,U) + \lambda G(\tau,\chi,U), \qquad \lambda \geq 0,$$

with λ reduced toward 0 as the run proceeds. The final draft is still evaluated by Hamming error; G is a steering signal.

A2.1. Add impact-aware initializations using η_{ij}-weighted row and column distances.

A2.2. Use impact residual to break exact Hamming ties in the tie-up update, row update, and column update.

A2.3. When using beam search, keep the best k assignment states under S_λ or under lexicographic (L_0, G) order, rather than following only one alternating path from each initialization.

A2.4. After a run terminates, compute the residual error pattern Δ and restart with row or column clusters biased toward high-impact residual regions.

A2.5. After converting a reduced triple into draft matrices, attempt local repairs near high-impact residual errors. A repair may be accepted according to S_λ during search, but final candidate selection remains Hamming-first.

These choices deliberately sacrifice the per-step monotonicity guarantee of Alpha 1 when $\lambda > 0$ is allowed to influence non-tied moves. That is a design choice: Alpha 2 is an optimizer to be measured, not a proof device. It should be compared with Alpha 1 by optimum-hit rate, average optimality gap, and runtime on benchmark instances.

3.3 Alpha 3

Alpha 3 adds a second search pressure based on the integer redundancy matrix

$$D = AB^TC.$$

The visible drawdown only records whether $D_{ij} > 0$, so an entry $D_{ij} > 1$ is not itself a Hamming error when $D^*_{ij} = 1$. It does, however, mean that multiple pressed treadles are doing redundant work at the same visible cell. The hypothesis behind Alpha 3 is that high redundancy is often correlated with inefficient local optima: the draft spends treadle activations overlapping cells already covered, leaving less flexibility for covering missing 1's without creating false positives.

Redundancy score

For a completed draft, define the redundancy pressure

$$R(D, D^*) = \sum_{i,j:D^*_{ij}=1} \max(0, D_{ij} - 1).$$

Equivalently, one may track per-pick redundancy

$$R_i(D, D^*) = \sum_{j:D^*_{ij}=1} \max(0, D_{ij} - 1).$$

False positives, where $D^*_{ij} = 0$ but $D_{ij} > 0$, are already counted by Hamming error; the redundancy score measures extra activation on cells that are visibly correct.

Redundancy-guided search pressure

Alpha 3 uses the same final objective $\Phi = \mathrm{err}$, but may guide draft construction and repair with

$$S_{\lambda,\mu} = \mathrm{err}(\widehat{D}, D^*) + \lambda G(\widehat{D}, D^*) + \mu R(D, D^*), \qquad \lambda, \mu \geq 0.$$

Here

$$G(\widehat{D}, D^*) = \sum_{i=1}^{p} \sum_{j=1}^{w} \eta_{ij} \, \mathbf{1}\{\widehat{D}_{ij} \neq D^*_{ij}\}$$

is the impact residual from Alpha 2 evaluated on the completed drawdown, and R is the redundancy pressure. In an annealed version, λ and μ are reduced toward 0 so that final selection returns to raw Hamming error.

Alpha 3 can use redundancy in several places.

A3.1. During conversion from reduced co-clusters to draft matrices, prefer treadle-generator decompositions that cover required 1's with less overlap.

A3.2. In beam search, rank draft-level states by $S_{\lambda,\mu}$, or by lexicographic order beginning with Hamming error and then redundancy.

A3.3. For picks with large R_i, try removing, replacing, or splitting pressed treadles. Such moves are useful when they preserve the visible drawdown while creating room for later repairs.

A3.4. Bias restarts toward rows or treadle combinations where redundancy is high but Hamming error remains nonzero nearby.

Alpha 3 is therefore not claimed to be pointwise better than Alpha 2. It is a more aggressive optimizer that treats redundancy as evidence of inefficient coverage. Its justification is empirical: if the redundancy pressure is useful, Alpha 3 should reduce the average optimality gap or improve the optimum-hit rate on small instances where exact reduced brute force is available.

3.4 Benchmarks

The benchmark tables below are reserved for empirical comparisons of Alpha 1, Alpha 2, and Alpha 3 on random binary targets. Rows list the algorithm, and columns list the draft size $p \times w$.

Table 1: Benchmark template for a standard 4-shaft loom with 4 shafts and 6 treadles.

Algorithm	10×10	20×20	30×30	40×40
Alpha 1				
Alpha 2				
Alpha 3				

Table 2: Benchmark template for a standard 8-shaft loom with 8 shafts and 10 treadles.

Algorithm	10×10	20×20	30×30	40×40
Alpha 1				
Alpha 2				
Alpha 3				

References

[1] Janice R. Lourie. Loom-constrained designs: An algebraic solution. In *Proceedings of the 1969 24th National Conference*, ACM, pages 185–192, 1969.

[2] J. Hoskins and W. Hoskins. The solutions of certain matrix equations arising from the structural analysis of woven fabrics. *Ars Combinatoria*, 11:51–59, 1981.

[3] J. Hoskins and W. Hoskins. A faster algorithm for factoring binary matrices. *Ars Combinatoria*, 16-B:341–350, 1983.

[4] J. Hoskins, A. Street, and R. Stanton. Binary interlacement arrays and how to find them. *Congressus Numerantium*, 42:321–376, 1984.

[5] J. Hoskins, C. Praeger, and A. Street. Twills with bounded float length. *Bulletin of the Australian Mathematical Society*, 28:255–281, 1983.

[6] Branko Grünbaum and G. C. Shephard. Satins and twills: An introduction to the geometry of fabrics. *Mathematics Magazine*, 53(3):139–161, 1980.

[7] Branko Grünbaum and G. C. Shephard. A catalogue of isonemal fabrics. In J. E. Goodman, editor, *Discrete Geometry and Convexity*, Annals of the New York Academy of Sciences, volume 440, pages 279–298, 1985.

[8] Branko Grünbaum and G. C. Shephard. An extension to the catalogue of isonemal fabrics. *Discrete Mathematics*, 60:155–192, 1986.

[9] Branko Grünbaum and G. C. Shephard. Isonemal fabrics. *The American Mathematical Monthly*, 95(1):5–30, 1988.

[10] Richard L. Roth. The symmetry groups of periodic isonemal fabrics. *Geometriae Dedicata*, 48:191–210, 1993.

[11] R. S. D. Thomas. Isonemal prefabrics with only parallel axes of symmetry. *Discrete Mathematics*, 309(9):2696–2711, 2009.

[12] R. S. D. Thomas. Isonemal prefabrics with perpendicular axes of symmetry. *Utilitas Mathematica*, 82:33–70, 2010.

[13] R. S. D. Thomas. Isonemal prefabrics with no axes of symmetry. *Discrete Mathematics*, 310(8):1307–1324, 2010.

[14] C. R. J. Clapham. When a fabric hangs together. *Bulletin of the London Mathematical Society*, 12:161–164, 1980.

[15] Cathy Delaney. When a fabric hangs together. *Ars Combinatoria A*, 21:71–79, 1986.

[16] T. C. Enns. An efficient algorithm determining when a fabric hangs together. *Geometriae Dedicata*, 15:259–260, 1984.